

Agentic API: A Multi-Agent System for CNPq Researcher Data

David Freitas¹, Melissa Victor²

¹ IC, UFBA, Salvador, Brazil.

² IC, UFBA, Salvador, Brazil.

Keywords: Multi-agent systems; Large Language Models; CNPq; Lattes Platform; Retrieval-Augmented Generation.

Abstract. This paper presents Agentic API, a multi-agent system for collecting, enriching, visualizing, and querying data about CNPq productivity scholarship holders in Computer Science. The pipeline deterministically extracts CNPq and Lattes data and uses LLM agents for ambiguity resolution, semantic inference, output repair, and natural language interaction. The enriched dataset is exposed through a FastAPI backend and a Next.js/React frontend supporting dashboards, researcher search, CSV export, and chat. The documented run processed 480 scholarship holders, matched 480 Lattes profiles, extracted 480 full curricula, and successfully repaired eight invalid inference outputs, leaving no unresolved inference errors in the final dataset.

1 Introduction

Academic research institutions generate large volumes of publicly available data. However, this information is often distributed across different platforms and is rarely integrated in a way that supports aggregated analysis, visualization, and natural language querying.

In Brazil, CNPq maintains public records of productivity scholarship holders, while the Lattes Platform provides a national repository of academic curricula [1, 2]. The CNPq Productivity Research Scholarship Program (*Bolsas de Produtividade em Pesquisa*, modalidade PQ) is an important instrument for recognizing and fostering scientific activity in Brazil.

The program organizes scholarships into different levels according to CNPq evaluation criteria, including scientific production and research leadership [3].

Although these sources contain complementary information, CNPq scholarship records are not directly integrated with the corresponding Lattes profiles. Both platforms are primarily designed for individual and manual navigation rather than aggregated analytical exploration. This limitation is particularly relevant in research management, where faculty coordinators, department heads, and funding agencies require queryable views of researcher profiles, expertise areas, career stages, and scientific production to support collaboration, reporting, and resource allocation.

To address this gap, we propose a system that automatically collects scholarship data from the public CNPq productivity scholarship table for Computer Science, identifies and matches each researcher with a Lattes profile through browser automation, retrieves the full curriculum, and applies a hybrid enrichment strategy combining deterministic rules with Large Language Model (LLM) inference. The resulting structured semantic profiles are exposed through a FastAPI-based REST API and support dashboard visualization, researcher search, data export, natural language queries, and auditable review artifacts.

The goals of this project are: (I) to generate a structured dataset of CNPq productivity scholarship holders enriched with Lattes information; (II) to provide interactive visualization and researcher search capabilities; (III) to support data export and natural language queries; and (IV) to reduce manual intervention while preserving traceability through logging and a multi-agent architecture. These goals are detailed in Table 1.

The remainder of this paper is organized as follows. Section 2 presents related work; Section 3 describes the proposed solution and architecture; Section 4 details the implementation; Section 5 presents the validation process; Section 6 discusses lessons learned and limitations; and Section 7 concludes the paper.

2 Related Work

Early tools such as scriptLattes [4] and LattesMiner [5] showed that it is possible to extract structured information from Lattes curricula and generate reports and datasets. However, both approaches depend on direct access to curricula, which limits their applicability under the current access restrictions of the Lattes Platform. More recent systems adopted different strategies. Science Tree [6] uses Lattes data to build Brazilian academic genealogy networks.

At the same time, instruction-following LLMs have increased interest in LLM-based multi-agent systems (LLM-MAS), in which complex tasks are divided among specialized agents that work together. Studies by Guo et al. [7], Hong et al. [8], and Li et al. [9] identify or implement task decomposition, centralized coordination, and result aggregation as key design patterns.

Recent studies have also shown the potential of LLM-based multi-agent systems for coordinating complex tasks. Guo et al. [7] discuss the importance of agent coordination, task decomposition, and communication, while MetaGPT [8] implements a structured workflow in which specialized agents collaborate across multiple stages. Furthermore, the implemented natural language interface of agentic-api follows the principles of Retrieval-Augmented Generation (RAG) proposed by Lewis et al. [10] and later categorized by Gao et al. [11], combining local retrieval over structured researcher profiles with LLM-based response generation.

Overall, previous studies demonstrate the feasibility of extracting and analyzing Lattes data, using multi-agent systems, and applying retrieval-augmented generation. To the best of our knowledge, the reviewed approaches do not combine public, unauthenticated extraction from the Lattes ecosystem, cross-source integration between CNPq records and Lattes profiles, semantic profile enrichment using coordinated LLM agents, and retrieval-based natural language querying. These limitations motivate the design of agentic-api.

3 Solution

3.1 Requirements

The project requires a multi-agent system capable of collecting, enriching, visualizing, exporting, and querying data about CNPq productivity scholarship holders in Computer Science. The source data is publicly available through the CNPq scholarship results page [2]. In addition to the functional requirements presented in Table 1, the system must preserve auditability, control LLM usage, retain intermediate artifacts, and keep ambiguous cases available for review rather than silently accepting uncertain results.

3.2 Architecture

The architecture consists of two major parts: an administrative data preparation pipeline that builds the enriched dataset and an application layer that exposes the active dataset to users. Because some stages depend on external web sources and LLM API calls, the pipeline has online dependencies. Its heavier collection, matching, enrichment, and validation operations are executed outside the application’s normal request–response flow. The application layer consumes the resulting active dataset to provide dashboard metrics, researcher search, export, and natural language interaction.

3.2.1 Pipeline Architecture

The data preparation pipeline is a staged workflow that transforms fragmented public sources into an enriched and auditable dataset. Its central principle is to collect factual data deterministically, use LLM agents only where interpretation or decision-making is required, and publish only a validated run as the active dataset.

Each stage has a well-defined responsibility and passes an explicit artifact or decision to the next stage. The workflow begins with CNPq scholarship extraction, proceeds through Lattes candidate matching and full curriculum collection, and then performs semantic inference, repair, normalization, review, and dataset promotion. This design avoids dependence on a

single opaque model call and allows individual stages to be inspected or reprocessed independently.

The complete system workflow is divided into two stages: data preparation and enrichment, shown in Figure 1, and dataset consumption and natural language querying, shown in Figure 2. Both figures are presented in Appendix A, while implementation details are discussed in Section 4.

3.2.2 Multi-Agent Architecture

The system consists of 16 specialized agents and processes, as summarized in Table 2. The architecture intentionally distinguishes deterministic components from LLM-based agents. For example, factual collection is performed by scrapers, whereas ambiguity resolution and semantic interpretation are delegated to LLM agents.

Inference generation and inference repair are represented as separate responsibilities because they use different models and are activated at different decision points. In this project, an agent is therefore defined by its responsibility, input, tools, output, and handoff rather than solely by whether it uses an LLM. This interpretation follows the broader multi-agent principle of decomposing complex tasks into specialized and coordinated responsibilities [7, 8].

3.2.3 Handoffs

A handoff defines the contract between two responsibilities: one component produces an artifact or decision, and another component consumes it. Explicit handoffs make the workflow inspectable and reduce hidden dependencies between stages. The main handoffs between agents and processes are summarized in Table 3.

The same principle is applied to the conversational workflow. The planner does not answer the user directly; it selects the tools required for the question. Likewise, the answer agent does not inspect the entire system freely. Instead, it receives a compact context package containing

retrieved or computed evidence and produces a grounded response.

3.2.4 Tools

The system uses conventional APIs and local tools rather than a dedicated MCP server. Its main technologies include Playwright and Chromium for browser automation, BeautifulSoup for HTML parsing, CSV and JSON for auditable storage, FastAPI for backend services, the OpenAI Responses API for LLM calls, File Search and Vector Store for optional semantic retrieval, and Next.js/React for the frontend. Table 4 summarizes these tools and their responsibilities.

A custom MCP server was not required because the available operations are local and controlled, including reading JSON files, filtering profiles, computing metrics, executing scraping stages, and invoking LLMs. MCP remains a possible extension if future versions require integration with external or independently hosted tools.

3.2.5 Guardrails and Reasoning

The main guardrails derive from the separation between factual collection and semantic interpretation. LLMs do not replace CNPq data and cannot create arbitrary Lattes candidates; ambiguity resolution is restricted to candidates collected from the source system. Exact counts, filters, and rankings are computed by deterministic backend tools, while Vector Store and File Search are reserved for semantic retrieval.

The planner cannot execute arbitrary code and may only request supported tools, fields, and operators. Raw HTML, text, CSV, and JSON artifacts are preserved for auditing, failed runs do not replace the active dataset, and inferred fields include confidence, source, reason, and review indicators. Sensitive fields such as `sex_inferred` are explicitly treated as approximate and are intended only for aggregate analysis.

Reasoning is concentrated in the natural language interface, which follows a planner-tool-answer workflow. The Query Planner Agent

selects structured queries, topic search, dashboard metrics, File Search, or a combination of these tools. Deterministic backend tools then produce an evidence package, which is passed to the Query Answer Agent. This hybrid approach follows Retrieval-Augmented Generation principles [10, 11] while using structured queries whenever exact numerical results are required.

4 Implementation

The system is implemented as a Python FastAPI backend with a Next.js/React frontend. The backend follows a layered structure: `app/routers` exposes the application interface, `app/services` contains business logic, and `app/scrapers` implements CNPq collection, Lattes processing, inference, and normalization. Generated datasets, manifests, and review artifacts are stored under `scrape_results`, while execution logs are stored under `logs`.

4.1 Data Preparation Pipeline

The pipeline begins with the `orchestrator_agent`, which creates a timestamped run, executes each stage in sequence, records logs, validates the result, and promotes the run only when it is usable. The `collector_scraper` extracts researcher names, scholarship levels, validity dates, institutions, and statuses from the public CNPq table. This stage is deterministic because the source already provides structured factual data.

The resulting records are passed to the `lattes_preview_agent`, which searches the Lattes interface and collects candidate profiles. Reliable matches proceed automatically, whereas ambiguous cases are sent to the `lattes_review_llm`. This agent is constrained to select only among collected candidates and cannot invent a Lattes profile or identifier.

Once a candidate is resolved, the `lattes_full_scraper` retrieves the complete curriculum and stores raw HTML, cleaned text, and structured JSON. These artifacts preserve the source evidence and support inference, de-

bugging, and reprocessing.

The `inference_agent` uses the model `gpt-5-nano` to transform curriculum evidence into semantic fields such as research areas, topics, methods, application domains, career indicators, experience attributes, and profile summaries. Its output is validated against the expected schema. Invalid or incomplete responses are forwarded to the `inference_repair_agent`, which uses `gpt-5.4-mini` to repair the result rather than discarding the profile.

After inference, deterministic normalization corrects label variations, institution-to-UF mappings, regions, and scholarship groupings. Profiles with uncertain `sex_inferred` values may be reviewed using names and gender-marked expressions found in the curriculum text. This field is treated as an approximation for aggregate visualization, not as an official demographic record.

The final dataset is stored in CSV and JSON formats. CSV supports inspection and export, while JSON preserves nested semantic fields. Each semantic field records its value, confidence, source, reason, and review status. The active dataset is selected through `scrape_results/current.json`; therefore, the application consumes a consistent snapshot instead of discovering files by scanning run directories. If a new run fails validation, the previously active dataset remains available.

4.2 Search and Natural Language Query Process

After dataset promotion, the `search_context_agent` generates `profiles_search_corpus.json` for semantic retrieval and compact context files for lightweight queries. This creates three complementary context levels: the local JSON dataset for exact structured operations, a minimal profile context for global reference, and an optional Vector Store corpus for richer semantic retrieval.

For each natural language question, the planner receives the available tools, dataset structure, dashboard context, recent conversation

history, and minimal profile context. It returns a structured plan specifying the required queries and filters. Backend tools execute the plan and construct an evidence package. The answer agent then explains the result using this evidence and avoids presenting unsupported claims as factual conclusions.

4.3 Application Layer and Auditability

The backend computes dashboard metrics before sending them to the frontend, avoiding the need to transfer the complete dataset to the browser. Available analyses include scholarship, institutional, regional, research-area, doctorate-year, and inferred-attribute distributions, as well as cross-tabulations useful for scholarship evaluation.

Profile search supports free-text search and filters by researcher, institution, scholarship level, region, inferred sex, research area, and topic. It also provides access to public Lattes profiles and CSV export. Search normalization handles accents, underscores, and multilingual variants such as *robotica* and *robotics*.

The frontend remains a consumption layer: it renders authentication, navigation, dashboards, profile search, and chat, but does not execute scraping, inference, or official metric calculations. Auditability is supported through stage summaries, retry logs, LLM review files, inference records, review queues, normalization logs, sex review logs, chat sessions, and Architecture Decision Records.

5 Testing

Validation was performed incrementally. CNPq extraction was first verified through generated tables and CSV files. Lattes matching was tested on one researcher, then on subsets, and finally on all 480 profiles. Full curriculum extraction was inspected through raw HTML, cleaned text, and structured JSON. Semantic inference was initially tested on small samples, after which the repair mechanism was added to handle invalid model outputs.

Dashboard metrics were checked against the active JSON dataset, while the chat was tested with exact counts, filters, topic searches, and

compound questions. The documented run processed 480 profiles, matched 480 Lattes previews, and extracted 480 full curricula. During semantic inference, eight invalid LLM outputs were detected and successfully repaired, resulting in zero unresolved inference-output errors in the final dataset.

These results validate the technical workflow but do not imply that every inferred field is perfectly accurate. Uncertain values remain visible through confidence scores, review flags, evidence artifacts, and logs.

6 Lessons Learned

6.1 Good points

A central design decision was separating factual extraction from semantic reasoning. Deterministic scrapers for CNPq and Lattes produce verifiable artifacts, while LLMs are reserved for tasks requiring interpretation, such as inference, ambiguity resolution, and summarization. This separation improves debuggability, transparency, and reproducibility while reducing the risk of hallucinated factual data. The system is also modeled as a set of responsibilities with explicit handoffs rather than as a collection of interchangeable LLM agents, allowing deterministic processes, backend services, and language models to coexist in a unified multi-stage pipeline with clear roles and output contracts. The staged pipeline further improves reliability because each stage can be executed and inspected independently. Earlier artifacts remain preserved even when later stages fail, simplifying fault isolation and reprocessing. Reproducibility is also reinforced by the dataset manifest, `current.json`, which separates experimental runs from the active dataset consumed by the application.

Finally, the planner–tool–answer chat architecture proved more reliable than a single LLM call. The planner selects the appropriate tools, the backend performs deterministic or retrieval-based computations, and the answer model produces a grounded response. This decomposition improves correctness for structured queries such as counts, rankings, and filtered searches.

6.2 Drawbacks

Despite its effectiveness, the system has several limitations. First, the pipeline is dependent on external HTML structures from CNPq and Lattes, meaning that changes in page layout may require updates to scraping logic. Second, the chat system introduces additional computational cost because each query typically involves multiple model calls. Although this improves quality and reliability, it also increases latency and operational cost. Third, semantic inference is currently limited to profile-level attributes. Although the system extracts research areas, topics, methods, and career indicators, it does not yet fully exploit publication-level data. Thus, information such as article titles, venues, co-authorship networks, and publication timelines remains underutilized, limiting deeper bibliometric and temporal analysis. Finally, although the current natural language toolset is functional, it does not yet cover all interaction scenarios and can be further expanded.

6.3 Future Improvements

Future work includes extending publication-level inference with bibliometric information such as articles, venues, co-author networks, and temporal publication patterns to support richer analyses of research evolution and collaboration. The natural language system could be expanded with a dedicated full-CV inspection tool for more detailed per-researcher queries. Infrastructure improvements include replacing local JSON/CSV storage with a relational database, introducing background job processing, and adding production-grade authentication and centralized observability. Future versions may also reduce chat costs through adaptive context selection, improve semantic search using local embeddings, expand human review interfaces, and integrate external knowledge sources such as Google Scholar and standardized tool orchestration protocols when multi-system integration becomes necessary.

7 Conclusion

This work presented Agentic API, a multi-agent system for collecting, enriching, visualiz-

ing, and querying data about CNPq productivity scholarship holders in Computer Science. The system integrates deterministic extraction from CNPq and Lattes with LLM-assisted ambiguity resolution, semantic inference, output repair, and natural language interaction.

The adopted hybrid architecture separates factual collection from semantic reasoning. Scrapers preserve source evidence, deterministic backend tools compute exact metrics and structured queries, and LLM agents handle tasks that require interpretation. This separation improves auditability, reproducibility, and reliability compared with an approach based exclusively on language models.

The documented execution processed 480 scholarship holders, matched 480 Lattes profiles, extracted 480 full curricula, and completed semantic inference with zero final inference errors after eight successful repairs. The resulting dataset supports dashboard analysis, researcher search, CSV export, and natural language queries.

Although the system remains dependent on external page structures and inferred fields require appropriate caution, it provides a functional and extensible foundation for research data analysis. Future work may expand publication-level analysis, human review interfaces, external data integration, and production-oriented infrastructure.

References

- [1] Conselho Nacional de Desenvolvimento Científico e Tecnológico. Plataforma Lattes. <http://lattes.cnpq.br/>, 2026. Accessed: 2026-06-22.
- [2] Conselho Nacional de Desenvolvimento Científico e Tecnológico. Resultados dos CAs: Bolsas de Produtividade em Pesquisa em Ciencia da Computacao. http://plsql1.cnpq.br/divulg/RESULTADO_PQ_102003.prc_comp_cmt_links?V_COD_DEMANDA=200310&V_TPO_RESULT=CURSO&V_COD_AREA_CONHEC=10300007&V_COD_CMT_ASSESSOR=CC, 2026. Accessed: 2026-06-22.
- [3] Conselho Nacional de Desenvolvimento Científico e Tecnológico. Bolsas de Produtividade em Pesquisa. <https://www.gov.br/cnpq/>, 2026. Accessed: 2026-06-22.
- [4] Jesús P. Mena-Chalco and Roberto M. Cesar Junior. scriptLattes: An open-source knowledge extraction

- system from the Lattes platform. *Journal of the Brazilian Computer Society*, 15:31–39, 2009.
- [5] Alexandre D. Alves, Horacio H. Yanasse, and Nei Y. Soma. LattesMiner: A multilingual dsl for information extraction from the Lattes platform. In *Proceedings of the Compilation of the Co-Located Workshops on DSM'11, TMC'11, AGERE! 2011, AOOPEs'11, NEAT'11, and VMIL'11, SPLASH '11 Workshops*, pages 85–92, New York, NY, USA, 2011. Association for Computing Machinery.
 - [6] João Cota, Alberto Laender, and Raquel Prates. Science Tree: A platform for exploring the brazilian academic genealogy, 2021.
 - [7] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: A survey of progress and challenges, 2024.
 - [8] Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Ceyao Zhang, Zili Wang, Steven Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, and Chenglin Wu. MetaGPT: Meta programming for multi-agent collaborative framework, 2023.
 - [9] Yuan Li, Lichao Sun, and Yixuan Zhang. MetaAgents: Large language model based agents for decision-making on teaming. *Proceedings of the ACM on Human-Computer Interaction*, 9(2):1–27, May 2025.
 - [10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems, NeurIPS '20*. Curran Associates Inc., 2020.
 - [11] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey, 2024.

Table 1: Functional Requirements of the Proposed System

ID	Name	Description
FR1	Dataset generation	Automatic creation of a structured dataset from the CNPq public scholarship database, including researcher name, inferred sex, institution, state (UF), scholarship level, expertise area, PhD year and Lattes URL.
FR2	Dashboard	Interactive dashboard for visualization of collected and enriched researcher data.
FR3	Data export	Export functionality for enriched researcher data in CSV format.
FR4	Natural language interface	Interface allowing users to query the dataset using natural language.
FR5	Operation logging	Logging system to record all pipeline operations and agent interactions.
FR6	Multi-agent architecture	System architecture based on multiple specialized agents handling different pipeline stages.

Table 2: System components: agents and processes in the pipeline

#	Agent / Process	LLM	Model	Responsibility
1	orchestrator_agent	No	–	Coordinates pipeline execution, logging, and dataset promotion.
2	collector_scraper	No	–	Extracts the official CNPq table.
3	lattes_preview_agent	No	–	Searches Lattes, collects candidate previews, and resolves deterministic safe matches.
4	lattes_review_llm	Yes	gpt-5.4-mini	Resolves ambiguous Lattes candidates.
5	lattes_full_scraper	No	–	Downloads full Lattes curriculum data.
6	inference_agent	Yes	gpt-5-nano	Generates semantic fields from curriculum data.
7	inference_repair_agent	Yes	gpt-5.4-mini	Fixes invalid inference outputs after validation.
8	normalization_process	No	–	Normalizes labels, UF, region, and derived fields.
9	sex_review_agent	Yes	gpt-5.4-nano	Reviews uncertain inferred sex cases.
10	search_context_agent	No	–	Builds compact context and search corpus.
11	dashboard_agent	No	–	Computes dashboard metrics.
12	profile_search_agent	No	–	Lists and exports researcher profiles.
13	query_planner_agent	Yes	gpt-5.4-mini	Plans tools for natural language queries.
14	backend_tools	No	–	Executes structured queries and metrics.
15	query_answer_agent	Yes	gpt-5.4	Validates the evidence and produces the final answer.
16	chat_title_agent	Yes	gpt-5.4-nano	Generates short chat titles.

A Detailed Pipeline

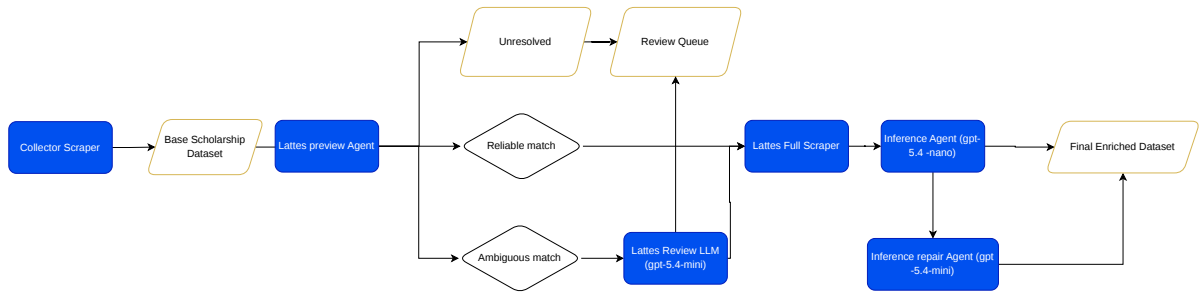


Figure 1: Data preparation and enrichment pipeline from CNPq collection to the final enriched dataset.

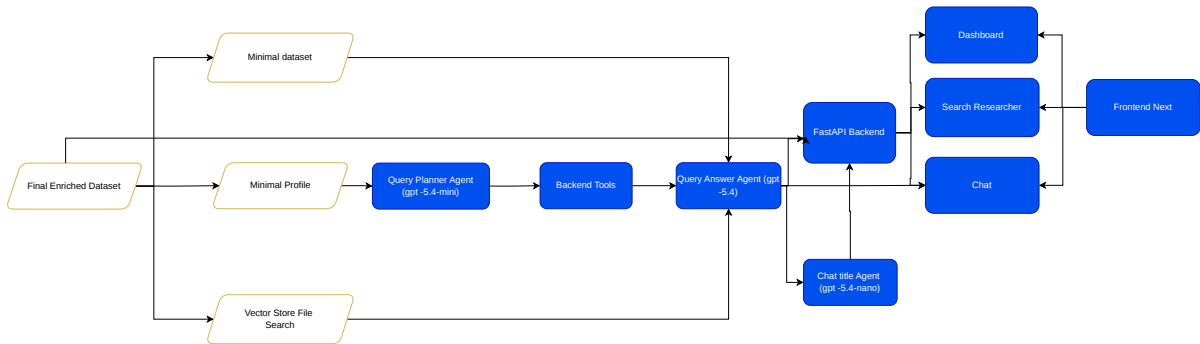


Figure 2: Application workflow from the active dataset to dashboard, profile search, and natural language querying.

Table 3: Data flow between agents and processes

From	To	Handoff artifact or decision
Collector Scraper	Lattes Preview Agent	Base scholarship records extracted from the CNPq table.
Lattes Preview Agent	Lattes Review LLM	Candidate list and review queue for ambiguous matches.
Lattes Review LLM	Lattes Full Scraper	Resolved Lattes profile when ambiguity can be safely solved.
Lattes Full Scraper	Inference Agent	Raw HTML, cleaned curriculum text, and structured profile JSON.
Inference Agent	Inference Repair Agent	Invalid or incomplete inference output requiring repair.
Inference / Repair	Normalization Process	Enriched profiles with semantic fields.
Normalization Process	Sex Review Agent	Profiles with unknown or low-confidence inferred sex.
Final Dataset	Dashboard / Profile Search / Chat	Active dataset (JSON/CSV) promoted via <code>current.json</code> .
Query Planner Agent	Backend Tools	JSON plan specifying which tools must be executed.
Backend Tools	Query Answer Agent	Compact context package with evidence and computed results.

Table 4: Core tools and technologies used in the system

Tool	Used by	Purpose
Playwright / Chromium	Scrapers and collectors	Accesses and extracts dynamically rendered web data from CNPq and Lattes.
BeautifulSoup	Parsing components	Extracts structured information from HTML pages.
CSV / JSON storage	All pipeline stages	Stores raw and processed data in an auditable format.
OpenAI API	LLM-based agents	Performs inference, ambiguity resolution, repair, planning, and response generation.
Vector Store / File Search	Search and chat agents	Enables retrieval over enriched datasets (RAG pipeline).
Structured Query Tool	Backend services	Executes deterministic queries, filters, and aggregations.
FastAPI	Backend system	Exposes API endpoints for pipeline, search, and chat services.
React / Next.js	Frontend	Provides user interface for dashboard, search, and chat.